

codehive

Python: Lists & Arrays

Foundation of Dynamic Data Management

1. Lists as Dynamic Arrays

In Python, the **List** is the primary built-in data structure used to implement arrays. Unlike static arrays in C++ or Java, Python lists are **dynamic**—they can grow and shrink in size automatically.

Key Traits: Ordered, Mutable (changeable), and Heterogeneous (can store integers, strings, and objects together).

2. Memory Allocation

When you add elements to a list, Python allocates more memory than strictly necessary. This "over-allocation" ensures that the next few `append()` operations are extremely fast.

This design makes `append()` an $O(1)$ amortized operation, meaning it's highly efficient for scaling data.

3. Basic Operations

Creating and initializing lists is straightforward using square brackets `[]`.

```
# Empty list
nums = []

# List with initial values
scores = [95, 88, 72, 91]

# Mixed types
data = [1, "Admin", 4.5, True]
```

4. Built-in Algorithms (Methods)

Python provides optimized built-in algorithms for manipulating lists:

- `append()`: Adds an item to the end.
- `sort()`: Rearranges items in ascending order.
- `pop()`: Removes the last item.
- `insert(idx, val)`: Adds an item at a specific position.

5. Creating Custom Algorithms

While built-ins are great, DSA is about writing your own logic. Consider the "Find Minimum" problem:

```
arr = [7, 12, 9, 4, 11]
min_val = arr[0]

for x in arr:
    if x < min_val:
        min_val = x
```

This simple logic is the foundation of more complex sorting algorithms.

6. Run Time Awareness

The time an algorithm takes to run is often proportional to the data size (N). In our "Find Minimum" example, to find the smallest value, Python must look at **every** element exactly once.

If you have 10 items, you check 10. If you have 1 billion, you check 1 billion. This relationship is crucial for optimization.

7. Linear Time Complexity $O(n)$

Because the work increases linearly with the data size, we call this **Linear Time** ($O(n)$).

Scaling Simulation

$N = 5 \rightarrow \text{Checks: } 5$

$N = 100 \rightarrow \text{Checks: } 100$

$N = 10^6 \rightarrow \text{Checks: } 1,000,000$

8. Optimization Strategies

Optimization is the art of reducing the number of operations. For example, if a list is **sorted**, finding the minimum value takes $O(1)$ time (it's just `arr[0]`), bypassing the need for a loop.

9. Practical DSA Exercises

Coding Challenge

1. Create a list of 10 random numbers.
2. Write a manual loop to find the **highest** number.
3. Calculate how many comparisons occurred.

10. Core Summary

- ✓ Lists are the "Dynamic Arrays" of Python.
- ✓ `append()` and `sort()` are built-in algorithms.
- ✓ Algorithm speed is relative to data size (N).
- ✓ Linear Search / Min-finding is **$O(n)$** .
- ✓ Optimization reduces unnecessary operations.

codehive

Arrays & Complexity Series | Module 02